

1 Course topics:

1. Finite automata / regular expressions
2. Context-free grammars
3. Turing Machines
4. Decidability / undecidability
5. NP-Completeness

2 Finite Automata

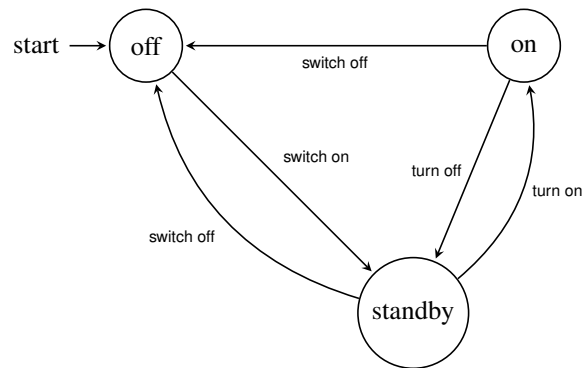
Automata: Plural of “automaton” (= machine)

A **Finite State Machine** is defined with:

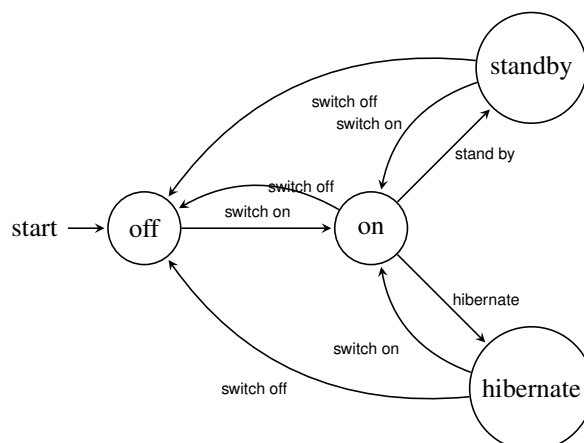
- Finite set of states
- Finite set of inputs
- Transition function
- A special start state

Examples:

TV:

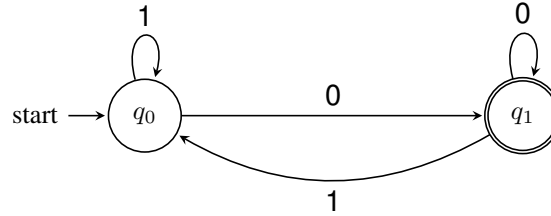


PC:



Finite Automaton is a Finite State Machine with some “accept state”.

M (q_1 is the accept state):



input string $w = 0100$ is *accepted* by M . M also accepts $0, 10, 110, 010, \dots$, i.e. all inputs that ends with 0.

2.1 Background Review

Some notations for sets A and B :

- $A \times B$: the *cartesian product* of A and B .
eg. $A = \{a, b\}$, $B = \{x, y\}$, then $A \times B = \{(a, x), (a, y), (b, x), (b, y)\}$
- A^k for integer k : $A \times A \times \dots A \times (k \text{ times})$
- 2^A : the *power set* of A .
eg. $A = \{a, b\}$; $2^A = \{\emptyset, \{a\}, \{b\}, \{a, b\}\}$
- $|A|$: number of elements in A (cardinality of A).

Some definitions:

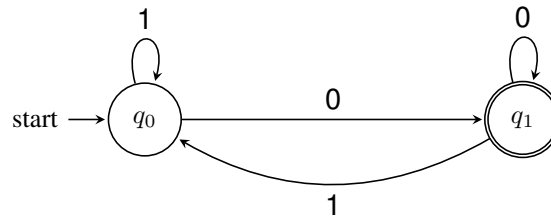
- alphabet: a finite set of symbols.
eg. $\Sigma_1 = \{0, 1\}$, $\Sigma_2 = \{a, b, c, \dots, z\}$
- string: a finite sequence of symbols; a.k.a. “word”.
eg. $w_1 = 0100$, $w_2 = abcbd$
- language: a set of strings over an alphabet.
eg. $L_1 = \{01, 10, 001\}$, $L_2 = \{w \in \{0, 1\}^* | w \text{ ends with } 0\}$

2.2 Deterministic Finite Automata

Definition: A deterministic finite automaton (DFA) is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where:

- Q is a finite set of states.
- Σ is a finite alphabet.
- $\delta : Q \times \Sigma \rightarrow Q$ is the transition function.
- $q_0 \in Q$ is the start state.
- $F \subseteq Q$ is the set of accept states.

Transition diagram representation:



Transition table representation:

Q/Σ	0	1
$\rightarrow q_0$	q_1	q_0
$*q_1$	q_1	q_0

Definition: A DFA M is said to accept an input string if its computation ends at an accept state.

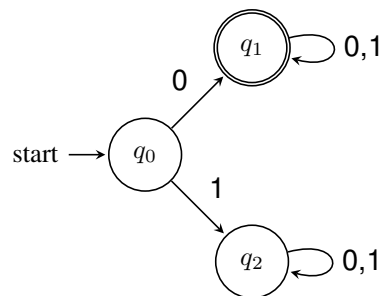
eg. M accepts 0100.

Definition: The language of a DFA M , $L(M)$, is the set of all input strings accepted by M .

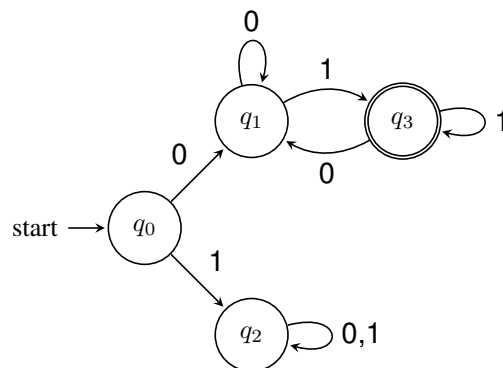
eg. $L(M) = \{w \mid w \text{ ends with } 0\}$

Examples:

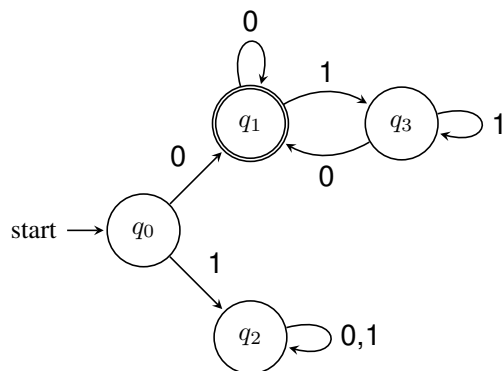
- $L : \{w \mid w \text{ begins with } 0\}$



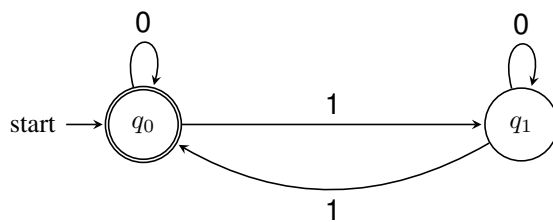
- $L : \{w \mid w \text{ begins with } 0 \text{ and ends with } 1\}$



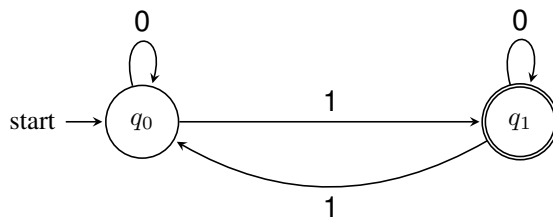
- $L : \{w \mid w \text{ begins and ends with } 0\}$



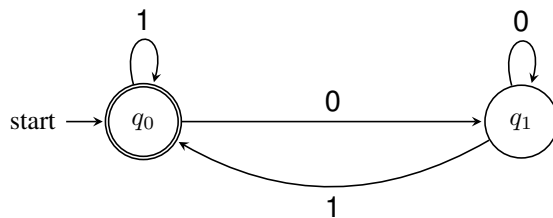
- $L : \{w \mid w \text{ has even number of } 1s\}$



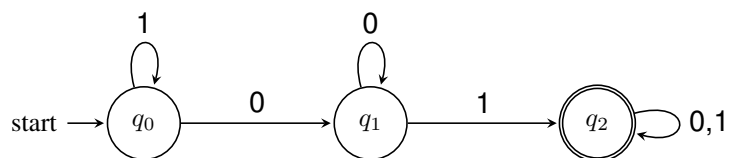
- $L : \{w \mid w \text{ has odd number of } 1s\}$



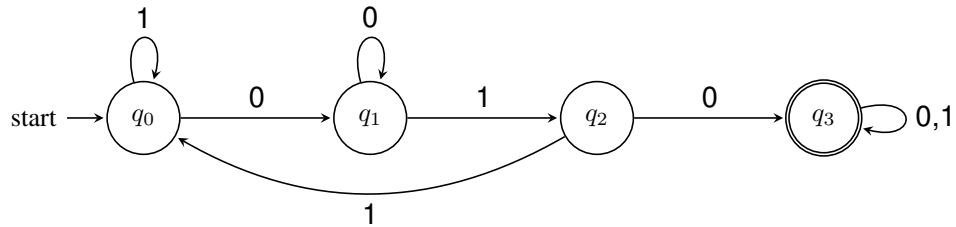
- $L : \{w \mid w \text{ ends with } 1 \text{ or } w = \epsilon\}$



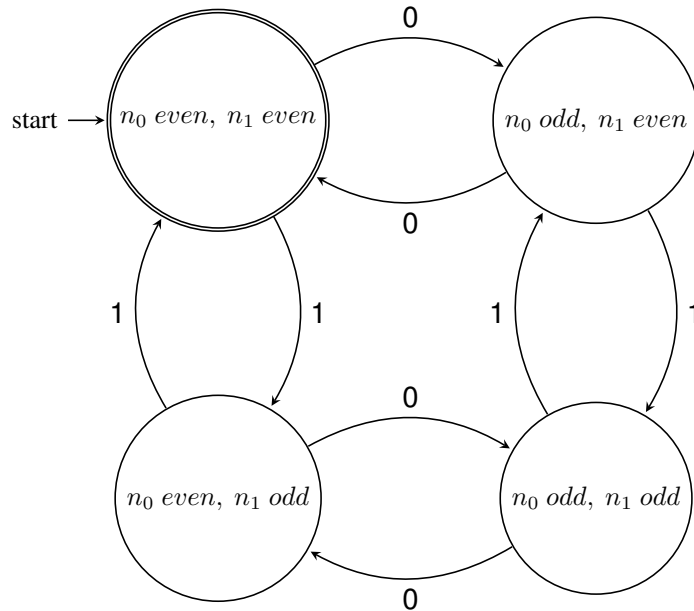
- $L : \{w \mid w \text{ contains } 01 \text{ as a substring}\}$



- $L : \{w \mid w \text{ contains } 010 \text{ as a substring}\}$



- $L : \{w \mid n_0(w) \text{ is even and } n_1(w) \text{ is even}\}$



- $L : \{w \mid n_0(w) \equiv n_1(w) \pmod{2}\}$
Same as above, but $n_0 \text{ odd}, n_1 \text{ odd}$ is also an accept state.
- $L : \{w \mid n_0(w) = n_1(w)\}$
Cannot be accepted by a DFA.

Definition: A language is *regular* if it is accepted by some DFA.

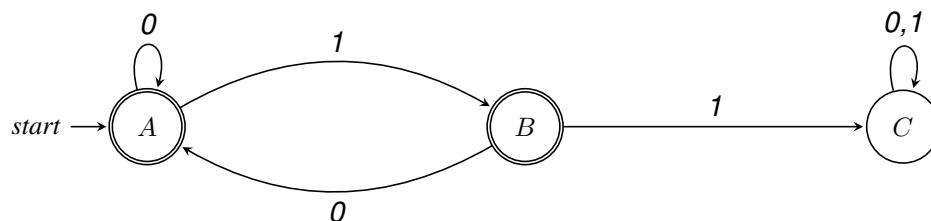
eg. $L_1 = \{w \mid w \text{ ends with } 0\}$

$L_2 = \{w \mid w \text{ contains an even number of } 0\text{s}\}$

...

Example:

Theorem 2.1 The language of the DFA M below is $L = \{w \mid w \in \{0,1\}^* \text{ and } w \text{ does not have two consecutive } 1\text{s}\}$.
 M :



Proof We have two sets S and T :

- S = “the language of M ”
- T = “the set of strings of 0s and 1s with no consecutive 1s”

To prove $S = T$, we need to prove both $S \subseteq T$ and $T \subseteq S$. That is:

- if $w \in S$, then $w \in T$.
- if $w \in T$, then $w \in S$.

Inductive hypothesis for Part 1 ($S \subseteq T$):

1. If $\delta(A, w) = A$, then w has no consecutive 1s and does not end in 1.
2. If $\delta(A, w) = B$, then w has no consecutive 1s and ends in a single 1.

Basis: $|w| = 0$; i.e. $w = \epsilon$

- (1) holds since ϵ has no 1s at all.
- (2) holds vacuously, since $\delta(A, \epsilon)$ is not B .

Induction:

- Assume (1) and (2) are true for strings shorter than w , where $|w| \geq 1$.
- Because $w \neq \epsilon$, we can write $w = xa$, where a is the last symbol of w , and x is the string that precedes.
- IH holds for x .
- Need to prove (1) and (2) for $w = xa$.
- (1) for w is: If $\delta(A, w) = A$, then w has no consecutive 1s and does not end in 1.
- Since $\delta(A, w) = A$, $\delta(A, x)$ must be A or B , and a must be 0 (look at the DFA).
- By the IH, x has no 11s.
- Thus, w has no 11s and does not end in 1.
- Now, prove (2) for $w = xa$: If $\delta(A, w) = B$, then w has no 11s and ends in 1.
- Since $\delta(A, w) = B$, $\delta(A, x)$ must be A , and a must be 1 (look at the DFA).
- By the IH, x has no 11s and does not end in 1.
- Thus, w has no 11s and ends in 1.

Inductive hypothesis for Part 2 ($T \subseteq S$):

- if w has no 11s, then w is accepted by M .
- **Contrapositive:** If w is not accepted by M , then w has 11.

Using the contrapositive:

- Because there is a unique transition from every state on every input symbol, each w gets the DFA to exactly one state.
- The only way w is not accepted is if it gets to C .
- The only way to get to C (formally: $\delta(A, w) = C$) is if $w = x1y$, x gets to B , and y is the tail of w that follows what gets to C for the first time.
- If $\delta(A, x) = B$ then surely $x = z1$ for some z .
- Thus, $w = z11y$ and has 11. ■

2.3 Nonregular Languages

Some languages are nonregular.

- $L_1 = \{0^n 1^n\}$ is nonregular since DFAs cannot count.
- $L_2 = \{w \mid w \in \{(\,,\,)\}^* \text{ and } w \text{ is balanced.}\}$. Balanced parentheses are those sequences of parentheses that can appear in an arithmetic expression. E.g.: $()$, $()()$, $((\,))$, $((\,))()$. . .

You can use CFGs to represent L_1 and L_2 above.

2.4 Regular Languages

They appear in many contexts and have many useful properties. Examples:

$L_3 = \{w \mid w \in \{0, 1\}^* \text{ and } w, \text{ viewed as a binary integer is divisible by } 23\}$

DFA M_3 to recognize L_3 :

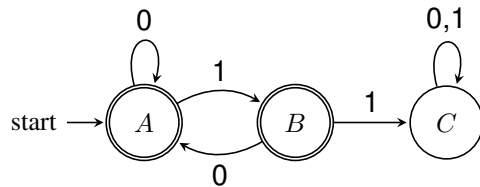
- 23 states, named $0, 1, \dots, 22$ that correspond to the 23 remainders of an integer divided by 23.
- Start and only final state is 0.
- If string w represents integer i , then assume $\delta(0, w) = i \bmod 23$.
- Then $w0$ represents integer $2i$, so we want $\delta(i \bmod 23, 0) = (2i) \bmod 23$.
- Similarly: $w1$ represents $2i + 1$, so we want $\delta(i \bmod 23, 1) = (2i + 1) \bmod 23$.
- Example: $\delta(15, 0) = 30 \bmod 23 = 7$; $\delta(11, 1) = 23 \bmod 23 = 0$.

Example 2:

$L_4 = \{w \mid w \in \{0, 1\}^* \text{ and } w, \text{ viewed as the reverse of a binary integer is divisible by } 23\}$

- 01110100 is in L_4 , because its reverse, 00101110 is 46 in binary.
- Hard to construct the DFA.
- But there is a theorem that says the reverse of a regular language is also regular.

2.5 Implementation of DFA



```

enum STATES {A, B, C} state;
int i;
char w[]; // from the user
state = A; // A is start state
for (i=0; i<strlen(w); i++){
    switch (state){
        case A:
            if (w[i]=='0') state=A;
            else state=B;
            break;
        case B:
            if (w[i]=='0') state=A;
            else state=C;
            break;
        case C:
            state=C;
            break;
    }
}
if (state == A || state == B){
    printf("accepted.\n");
}
else{
    printf("rejected.\n");
}
  
```

Recall $L_3 = \{w | w \in \{0,1\}^* \text{ and } w, \text{ viewed as a binary integer is divisible by } 23\}$

if $(w)_2 = (i)_{10}$; then:

$\delta(0, w) = i \bmod 23$. Thus

$Q = \{0, 1, \dots, 22\}$, $q_0 = \{0\}$ and $F = \{0\}$.

Note that $(w0)_2 = (2i)_{10}$ and $(w1)_2 = (2i + 1)_{10}$.

$$\delta(0, w) = i \bmod 23.$$

$$\delta(0, w0) = 2i \bmod 23.$$

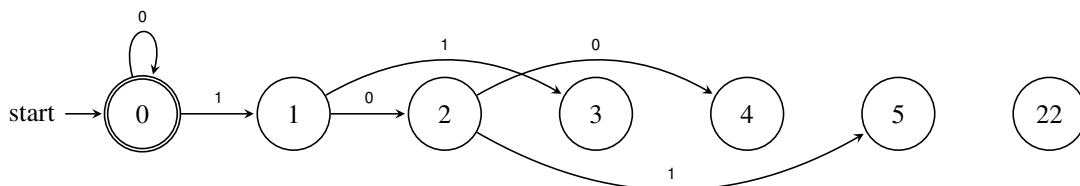
$$\delta(0, w0) = \delta(\delta(0, w), 0) = \delta(i \bmod 23, 0) = 2i \bmod 23.$$

Similarly :

$$\delta(0, w1) = (2i + 1) \bmod 23.$$

$$\delta(0, w1) = \delta(\delta(0, w), 1) = \delta(i \bmod 23, 1) = (2i + 1) \bmod 23.$$

DFA (partial):



For implementation, see

<http://www.cs.bilkent.edu.tr/~calkan/teaching/cs476/dfaMod.c>